Brickts SQL course — Week 2 — Design trustworthy tables

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 2 — Design trustworthy tables

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 08: Day 08 — Choose a grain and a dependable key
- Day 09: Day 09 — Create a small table with useful constraints

- Day 10: Day 10 — Separate facts and enforce relationships
- Day 11: Day 11 — Join metadata and preserve empty groups
- Day 12: Day 12 — Detect a join that multiplies observations
- Day 13: Day 13 — Make repeated ingest safe in a transaction
- Day 14 (weekly challenge): Day 14 — Week 2 challenge — design a small relational schema

---

Day 08 — Choose a grain and a dependable key

Previous: Day 7 · **Week 2 · 45–75 minutes.**
Prerequisites: days 1–7, or equivalent skills.

Goal

Describe a table's grain, distinguish candidate keys from convenient labels, and select data types that preserve meaning. Good queries begin with a clear answer to "one row per what?"

Before you start

Connect to the course database. Keep your first site report nearby. Inspect `course.site`, `course.equipment`, `course.point`, and `course.sample` with `\d`. You are reading definitions today; keep the fixture unchanged.

Concept

A primary key uniquely identifies a row and cannot contain nulls. A candidate key is any column set that could meet that identity requirement. A foreign key relates stored identities across tables. A label is intended for people and may change or repeat. Adding more descriptive text does not automatically create reliable identity.

The sample key `(site_id, point_id, ts)` describes one point observation at one instant within a site. Repeating a point at a different instant is legitimate; repeating the full key is not a second logical observation. The site component also makes ownership explicit.

Types constrain what a value can mean. `text` holds identifiers such as `SYN-NORTH`; `uuid` holds opaque point identities; `timestamptz` holds instants; `double precision` holds approximate measurements. Numeric measurements need

units and quality metadata to be useful. A type alone cannot distinguish pascals from degrees Celsius.

Tiny example

```
SELECT label, count(*) AS
         equipment_with_label
FROM course.equipment
GROUP BY label
ORDER BY label;
```

`Supply A` appears twice and `Supply B` once. Therefore label alone cannot be an equipment key. Inspect those two identities:

```
SELECT equipment_id, site_id, label
FROM course.equipment
WHERE label = 'Supply A'
ORDER BY site_id, equipment_id;
```

The two rows belong to different sites and have different UUIDs.

SQL challenge

Write a compact schema notebook with one row per course table describing its grain, primary key, important column types, and one relationship. Include `course.sample`. Verify

your descriptions against `\d` rather than inferring constraints from a few observed rows.

Write a query that groups points by site, equipment identity, and role and returns groups with more than one row. The starter fixture should return zero groups because it has one point per equipment and role. Explain why a future duplicate sensor would require a selection policy, even if both UUIDs remain unique.

Experiment

Compare grouping equipment by label with grouping by site and label. The latter happens to distinguish all three fixture AHUs. Does that observation prove the schema enforces uniqueness of site and label? Read its constraints before answering. Data observed today and rules guaranteed tomorrow are different evidence.

Acceptance checks

- Your notebook distinguishes 42 point identities from 24,190 sample identities.
- You demonstrate the repeated label without claiming label is a key.

- Each type explanation includes its meaning and one limit.

Stretch and reflection

When would a numeric-looking building code need `text`? Consider leading zeros and future letters. Read week 2 references for keys, types, and constraints.

Previous: Day 7 ·

Day 09 — Create a small table with useful constraints

Previous: Day 8 · **Week 2 · 45–75 minutes.**
Prerequisites: days 1–8, or equivalent skills.

Goal

Create a scratch table whose constraints reject a few clearly invalid states, and distinguish missing values from out-of-range values.

Before you start

Use only the `student` schema in `brickts_course` for new objects. Save the DDL in a file. DDL means data definition language: statements such as `CREATE TABLE` that define a database structure. If an object already exists from an earlier attempt, inspect it and choose another scratch name; do not delete unknown objects to clear an error.

Concept

A table definition is a contract for every writer. `PRIMARY KEY` prevents duplicate identities. `NOT NULL` requires a value. `CHECK` requires a Boolean condition that is not false, which means a null result can pass a check. Combine `NOT NULL` with a range check when both are required.

A useful constraint describes a durable rule, such as a percentage being within zero and one hundred. A short-term expectation, such as the current outdoor temperature, does not belong in a universal type rule. Database constraints protect stored states; they do not prove that a sensor was calibrated or that an operator chose the right point.

Tiny example

```
CREATE TABLE student.day09_setting (
    setting_id text PRIMARY KEY,
    fan_speed_pct double precision NOT
        NULL,
    CHECK (fan_speed_pct BETWEEN 0 AND
        100)
);

INSERT INTO student.day09_setting
        (setting_id, fan_speed_pct)
VALUES ('practice-1', 40);

SELECT setting_id, fan_speed_pct
FROM student.day09_setting;
```

One row should exist with `fan_speed_pct = 40`. The grain is one scratch setting per chosen identifier; this is not a command sent to equipment. Inspect the definition with `\d student.day09_setting`.

SQL challenge

Design a separate scratch table for a lab threshold configuration. Give it a text identity, a required role name, a required unit, and required lower and upper numeric bounds. Enforce that the lower bound does not exceed the upper bound. State the table's grain and which rules it guarantees.

Insert two valid rows with different identities. Test a reversed range, a repeated identity, and a missing required value in separate statements. Record which named or described constraint rejects each input. Keep failed tests separate so you can understand each diagnostic.

Experiment

Test `0` and `100` in the tiny percentage table, using fresh setting IDs. Both boundaries pass. Test `-1`, `101`, and `NULL`; all fail for reasons you should distinguish. Do these checks enforce a particular sampling unit or make an actual fan safe to operate? Explain their limited scope.

Acceptance checks

- Your DDL runs in `student`, and two valid threshold rows persist.
- Invalid bounds, null required fields, and duplicate keys are rejected with recorded evidence.
- Your explanation distinguishes a check from a not-null requirement.

Stretch and reflection

Read the meaning of finite floating-point values in week 2 references. The main fixture has additional finite-value protection. Explain why “is numeric” is not a complete measurement-quality policy.

Previous: Day 8 ·

Day 10 — Separate facts and enforce relationships

Previous: Day 9 · **Week 2 · 45–75 minutes.**
Prerequisites: days 1–9, or equivalent skills.

Goal

Recognize repeated facts, explain basic normalization, and use a foreign key to keep related scratch tables consistent.

Before you start

Review the grains and keys in your schema notebook. Today's scratch objects are independent of yesterday's table. All inserts stay in `student`; you do not need to add any course sites or equipment.

Concept

Suppose every sample row repeats a site's label and timezone. Renaming that label would require changing many rows, and missed updates could leave contradictory descriptions. Normalization places each fact

with the identity it describes: site facts with a site, equipment facts with equipment, and observations with point and instant.

Splitting tables is useful only when their relationships remain explicit. A foreign key requires a referenced key to exist. A composite foreign key checks a combination together. In the course, a point's equipment must belong to the same site as the point; checking only that an equipment UUID exists would not establish that ownership rule.

Tiny example

```
CREATE TABLE student.day10_site (
    site_id text PRIMARY KEY,
    label text NOT NULL
);
CREATE TABLE student.day10_equipment (
    equipment_code text PRIMARY KEY,
    site_id text NOT NULL
        REFERENCES student.day10_site
         (site_id),
    label text NOT NULL
);
INSERT INTO student.day10_site (site_id,
         label)
VALUES ('practice-site', 'Practice site');
INSERT INTO student.day10_equipment
         (equipment_code, site_id, label)
VALUES ('practice-ahu', 'practice-site',
         'Practice supply');
```

There is one parent and one child. The text equipment code is a scratch simplification; real course equipment continues to use its frozen UUID identity. A child with `site_id = 'unknown-site'` fails because no matching parent exists.

SQL challenge

Sketch a small normalized design for maintenance notes tied to equipment identity. Decide where equipment labels live and what one note row represents. Create a scratch notes table referencing `course.equipment(equipment_id)` and insert one note by selecting an equipment UUID from metadata. Your query must show the site and label used to choose that equipment, so the repeated label cannot cause an accidental choice.

Explain what happens if a note refers to an invented UUID. Test rejection using your scratch table and a clearly synthetic UUID; do not alter course metadata.

Experiment

Change the label of the practice site and read the child row. Its relationship still uses the stable site ID, so the equipment does not need

a new identity. Try inserting an equipment row before its missing practice parent exists. Record the foreign-key diagnostic and explain the required relationship.

Acceptance checks

- Your design stores each equipment label in its metadata location and stores note-specific facts with each note.
- A valid equipment reference is accepted; an unknown reference is rejected.
- You can explain why the course also checks site/equipment ownership together.

Stretch and reflection

What should happen to notes when equipment is retired? Compare rejecting deletion, retaining a retired metadata row, and cascading deletion conceptually. Do not test destructive policies on the fixture. Read week 2 references for referential integrity and normalization.

Previous: Day 9 ·

Day 11 — Join metadata and preserve empty groups

Previous: Day 10 · **Week 2 · 45–75 minutes.** Prerequisites: days 1–10, or equivalent skills.

Goal

Use an inner join to combine related facts and a left join to retain a parent that has no matching detail. Explain what one result row represents after each join.

Before you start

Revisit the fixture's relationships: site → equipment → point → sample. A join uses these stored identities to combine rows. Similar labels are not evidence that two rows describe the same entity.

Concept

An inner join emits matching pairs. If one equipment has fourteen points, joining it to points produces fourteen rows, each with repeated equipment facts. This repetition is expected because the result's grain is now a point.

A left join keeps each left-side row even when no right-side row matches, filling right-side columns with nulls. A filter on the right table placed in `WHERE` can remove these unmatched rows. Put a condition in `ON` when it defines which optional matches are eligible. Count a non-null child key to count matches: `count(*)` would also count the left join's preserved row.

Tiny example

```
SELECT e.equipment_id, e.site_id, e.label
         AS equipment_label,
       s.label AS site_label
FROM course.equipment AS e
JOIN course.site AS s ON s.site_id =
         e.site_id
ORDER BY e.site_id, e.equipment_id;
```

There are three rows, one per equipment identity. Joining site metadata does not multiply these rows because each site ID identifies one site.

A useful optional-detail report is:

```
SELECT s.site_id, count(h.point_id) AS
         bad_sample_rows
FROM course.site AS s
LEFT JOIN course.history AS h
  ON h.site_id = s.site_id AND h.quality =
         'bad'
GROUP BY s.site_id
ORDER BY s.site_id;
```

It returns north `1` and south `0`.

SQL challenge

Build a point inventory by joining `course.point` to `course.equipment` and `course.site`. Use point/equipment UUIDs and matching site IDs in the relationship. Show site ID, equipment identity and label, point identity, role, and unit. Predict and verify forty-two rows.

Then create a site report that counts matching `duct_static` sample rows while retaining every site. Explain which relation supplies the group identities and which key counts detail rows.

Experiment

Move `h.quality = 'bad'` from the tiny left join's `ON` to a `WHERE` clause. South disappears because its preserved row has null right-side values. Restore the condition to `ON` and replace `count(h.point_id)` with `count(*)`; south misleadingly becomes `1`. Describe both mistakes in terms of rows, not syntax memorization.

Acceptance checks

- Your point inventory has 42 distinct point identities and enough ownership context.
- The optional bad-quality report retains both sites with counts 1 and 0.
- You can explain the effect of moving a right-side filter after a left join.

Stretch and reflection

Read week 2 references for join types. Which questions need unmatched parents to remain visible, and which deliberately ask only for existing matches?

Previous: Day 10 ·

Day 12 — Detect a join that multiplies observations

Previous: Day 11 · **Week 2 · 45–75 minutes.** Prerequisites: days 1–11, or equivalent skills.

Goal

Recognize an accidental many-to-many join, measure its effect, and repair it using the relationship that matches your intended grain.

Before you start

Keep yesterday's point inventory available. The fixture has three points for each role, one on each AHU. Role names describe a measurement category; a role name alone does not identify a point or its owning equipment.

Concept

A join emits every pair that satisfies its condition. If three left rows and three right rows share the same role, joining only on role

creates nine pairs. SQL has done exactly what was requested, even if a report author expected three.

`DISTINCT` can hide repeated output values, but it cannot establish that the correct physical entities were paired. Aggregating after a mistaken join can silently inflate counts and sums. Test row counts and key uniqueness before trusting an engineering calculation. The correct join condition follows the relationship: sample identity to point identity with site ownership, then point's equipment identity to equipment.

Tiny example

```
SELECT p.point_id AS left_point,
         q.point_id AS right_point,
       p.role
FROM course.point AS p
JOIN course.point AS q ON q.role = p.role
WHERE p.role = 'duct_static'
ORDER BY p.point_id, q.point_id;
```

This deliberately incorrect identity match returns nine rows. Only three pair a point with itself. Inspect the UUIDs to see the six cross-equipment pairs instead of merely noting repeated role text.

SQL challenge

Write an explicit `course.sample` → `course.point` → `course.equipment` query for `kw` observations. Show site ID, equipment identity and label, point identity, timestamp, value, and quality. Count its rows in a separate query: 1,728 are stored, including one unusable power observation. Check that no `(site_id, point_id, ts)` group occurs more than once.

Now document a deliberately wrong version that joins a sample's point metadata to another point by role alone. Keep it bounded or count its rows; do not present it as a valid report. Explain how your corrected version retains the intended observation grain.

Experiment

Predict the wrong power-query row count. Since each original power row can match three role peers, expect 5,184 rows. Compare counts before and after the additional join. Why might an average happen to stay unchanged while the count triples? A plausible average is not proof of a correct relationship.

Acceptance checks

- The valid power join has 1,728 rows and no repeated logical sample key.
- Your evidence demonstrates the nine-pair point example and explains its cause.
- The fix uses declared identities and ownership, with no arbitrary first-row selection.

Stretch and reflection

Imagine two temperature sensors with the same role on one AHU. What metadata or explicit selection policy would a report need? Write the ambiguity as an unresolved requirement rather than choosing one by UUID order. Read week 2 references for join semantics and uniqueness.

Previous: Day 11 ·

Day 13 — Make repeated ingest safe in a transaction

Previous: Day 12 · **Week 2 · 45–75 minutes.** Prerequisites: days 1–12, or equivalent skills.

Goal

Use a unique key and `ON CONFLICT` to handle a repeated ingest attempt, then roll back a scratch change. Decide whether a repeat should be ignored or treated as a correction.

Before you start

All writes remain in `student`. Read `\d course.sample` to confirm its logical key, and keep the fixture unchanged. Today's scratch table is deliberately small; it demonstrates ingest behavior without copying all 24,190 observations.

Concept

An ingest process can retry after a timeout even when the first write succeeded. A key ensures that retrying cannot create a second

logical observation. `ON CONFLICT DO NOTHING` keeps the existing row. `DO UPDATE` can replace stored fields, but a correction policy must say which incoming record wins and why.

A transaction groups statements between `BEGIN` and `COMMIT`. `ROLLBACK` discards the transaction's uncommitted changes. A successful statement is not durable until its transaction commits. With ordinary `psql` autocommit enabled, an individual statement runs in its own transaction unless you explicitly start one.

Tiny example

```
CREATE TABLE student.day13_sample (
    site_id text NOT NULL,
    point_id uuid NOT NULL,
    ts timestamptz NOT NULL,
    value double precision,
    quality text NOT NULL CHECK (quality
         IN ('good','bad','missing')),
    PRIMARY KEY (site_id, point_id, ts)
);

INSERT INTO student.day13_sample (site_id,
         point_id, ts, value, quality)
SELECT site_id, point_id, ts, value,
         quality
FROM course.sample
ORDER BY ts, site_id, point_id
LIMIT 1
ON CONFLICT (site_id, point_id, ts) DO
         NOTHING;
```

Run the insert twice. The scratch table still contains one row. That row's key came from stored fixture identity; it was not invented from an equipment label.

SQL challenge

Create your own scratch ingest table with the logical observation key and documented null/quality policy. Insert a bounded ten-row fixture excerpt, repeat the same batch, and prove there are still ten rows. State whether your baseline accepts corrections or keeps the first stored value.

Test a correction inside a transaction, using the copied row's key and a visibly synthetic value. Observe it inside the transaction, then roll back and verify the original scratch value returns. Record the before, during, and after results.

Experiment

In `student.day13_sample`, run `BEGIN;`, change `quality` to `bad`, inspect it, and run `ROLLBACK;`. The table returns to its original quality. What would `COMMIT` have meant? If a statement fails inside a transaction, use `ROLLBACK` before continuing; the failed transaction cannot silently resume normal work.

Acceptance checks

- A repeated batch creates no duplicate logical rows.
- Your correction experiment changes scratch data temporarily and restores it with rollback.
- You explicitly distinguish retry handling from correction precedence.

Stretch and reflection

Could a late arriving bad record overwrite a newer good correction under a naive upsert? Describe the extra source/version metadata a real policy needs. Consult week 2 references for `INSERT`, conflict handling, and transactions.

Previous: Day 12 ·

Day 14 — Week 2 challenge — design a small relational schema

Previous: Day 13 · **Week 2 review · 2–4 hours.** Prerequisites: days 8–13.

Project brief

Design a reviewable scratch schema for an additional synthetic site, its equipment, point metadata, and a bounded observation excerpt. Use only `student` objects. Keep course metadata and samples intact. Work independently from a written statement of each table's grain; allow 2–4 hours.

Your design should let another learner understand ownership, units, quality, and repeated ingest without guessing from names. It may be smaller than the course schema, but explain every omitted rule. Use frozen synthetic identities and explicit relationships rather than deriving runtime identity from labels.

Acceptance criteria

- Each table has a stated grain and a key that identifies that grain.
- Foreign keys prevent unknown parent references, and ownership across sites is explained and enforced wherever your design stores both identities.
- Required values, valid quality states, and basic measurement/configuration checks have appropriate constraints.
- A repeated logical observation cannot create a duplicate; your correction policy is written down.
- A join report preserves the intended observation grain and distinguishes two equipment records with the same label.
- Evidence includes valid input, rejected invalid input, duplicate ingest, and a scratch rollback.

Deliverables

Submit DDL, bounded seed/ingest SQL, a schema sketch or relationship description, a report query, and a short transcript. Explain one design tradeoff, one rule enforced by the database, and one rule that still needs application or human judgment. Mark evidence observed or not run.

Review questions

Which repeated fact would cause inconsistent updates? How do you know a join did not multiply rows? What does a foreign key guarantee about ownership, and what extra combination might be needed? When should a retry change a value?

Use week 2 references after your independent attempt; no worked implementation is supplied here.

Previous: Day 13 ·

Primary reading — Week 2

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 2

- Data definition
- Constraints
- Table expressions and joins
- INSERT and conflict handling